LA MINUTE DE CODE



FORMATION: JAVASCRIPT DE A À Z

PARTIE 4

LA MINUTE DE CODE

PLAN DE LA FORMATION

JS

- Les promesses
- Async await
- API
- La méthode fetch

LA MINUTE DE CODE

LES PROMESSES

Les promesses en JavaScript sont un moyen d'effectuer des opérations asynchrones dans un code plus facilement et de manière plus lisible.

Une promesse est un objet qui représente une valeur qui peut ne pas être disponible immédiatement, mais qui sera résolue à une date ultérieure. Lorsqu'une opération asynchrone est effectuée, une promesse est créée pour représenter le résultat de l'opération. La promesse peut ensuite être manipulée de différentes manières pour traiter le résultat.

Une promesse a trois états possibles : "en attente" (pending), "résolue" (resolved), et "rejetée" (rejected). Une fois qu'une promesse est résolue ou rejetée, elle ne peut plus changer d'état.

Voici un exemple simple d'utilisation de promesses en JavaScript :

```
const myPromise = new Promise((resolve, reject) => {
  // Effectuer une opération asynchrone ici, par
exemple une requête AJAX
  // Si l'opération réussit, appeler resolve() avec
le résultat
  // Si l'opération échoue, appeler reject() avec
l'erreur
});

myPromise.then(result => {
  // Faire quelque chose avec le résultat de la
promesse
}).catch(error => {
  // Gérer l'erreur ici
});
```

Dans cet exemple, nous créons une nouvelle promesse en passant une fonction au constructeur Promise. Cette fonction prend deux arguments, resolve et reject, qui sont des fonctions à appeler en fonction du résultat de l'opération asynchrone.

Ensuite, nous chaînons deux méthodes sur la promesse : then pour traiter le résultat en cas de réussite et catch pour gérer les erreurs. Si la promesse est résolue avec succès, la fonction passée à then est appelée avec le résultat. Si la promesse est rejetée, la fonction passée à catch est appelée avec l'erreur.

Les promesses sont souvent utilisées avec les fonctions asynchrones, qui sont une syntaxe plus claire et concise pour les opérations asynchrones en JavaScript. Les fonctions asynchrones retournent des promesses et peuvent être appelées avec await, ce qui permet d'attendre que la promesse soit résolue avant de continuer l'exécution du code.

LA MINUTE DE CODE

ASYNC AWAIT

async/await est une syntaxe introduite dans ECMAScript 2017 pour rendre plus facile l'écriture de code asynchrone en JavaScript. Cette syntaxe permet d'écrire du code asynchrone de manière plus synchrone et plus lisible.

En JavaScript, les opérations asynchrones sont souvent gérées à l'aide de callbacks ou de promesses. Cependant, ces approches peuvent rendre le code difficile à lire et à comprendre, surtout lorsque plusieurs opérations asynchrones sont en jeu. async/await est une alternative à ces approches qui rend le code asynchrone plus facile à lire et à comprendre.

La syntaxe async/await se compose de deux éléments principaux :

async : Cette étiquette est utilisée pour déclarer une fonction asynchrone. Les fonctions asynchrones retournent toujours une promesse. Une fonction asynchrone peut contenir des expressions await.

await : Cette expression est utilisée à l'intérieur d'une fonction asynchrone pour attendre la résolution d'une promesse. Lorsque l'expression await est rencontrée, elle interrompt l'exécution de la fonction asynchrone jusqu'à ce que la promesse soit résolue. Ensuite, elle retourne la valeur résolue de la promesse.

LA MINUTE DE CODE

ASYNC AWAIT

JS

Voici un exemple simple pour illustrer l'utilisation de async/await :

```
async function fetchData() {
  const response = await fetch('https://example.com/
data');
  const data = await response.json();
  return data;
}

fetchData().then(data => {
  // Faire quelque chose avec les données
}).catch(error => {
  // Gérer l'erreur
});
```

Dans cet exemple, nous définissons une fonction asynchrone fetchData(). À l'intérieur de cette fonction, nous utilisons l'expression await pour attendre la résolution de deux promesses : la promesse retournée par la méthode fetch() et la promesse retournée par la méthode json().

Lorsque la fonction fetchData() est appelée, elle retourne une promesse qui sera résolue avec les données récupérées depuis l'URL spécifiée. Nous utilisons la méthode then() pour faire quelque chose avec les données une fois qu'elles sont récupérées.

Si une erreur se produit à n'importe quel moment dans la chaîne async/await, elle sera propagée à la promesse retournée par la fonction asynchrone, et la méthode catch() peut être utilisée pour gérer l'erreur.
En résumé, async/await est une syntaxe qui permet d'écrire du code asynchrone de manière plus synchrone et plus lisible. Elle est particulièrement utile lorsque plusieurs opérations asynchrones sont en jeu.

API

Les API (Application Programming Interfaces) en JavaScript sont des interfaces qui permettent aux développeurs d'interagir avec des services externes à travers des protocoles standardisés tels que HTTP. Les API peuvent fournir des données ou des fonctionnalités à d'autres applications ou services.

Pour effectuer un appel à une API en JavaScript, il est courant d'utiliser la méthode fetch(), qui est une méthode standard du navigateur pour récupérer des ressources à partir d'un serveur. Cependant, il est également possible d'effectuer des appels à des API sans utiliser fetch() en utilisant des techniques telles que l'objet XMLHttpRequest.

Voici un exemple d'une ancienne manière pour faire appel à une API en utilisant XMLHttpRequest :

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://api.example.com/data');
xhr.onload = function() {
  if (xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
    // Faire quelque chose avec les données
  } else {
    console.log('Erreur : ' + xhr.status);
  }
};
xhr.send();
```

Dans cet exemple, nous créons une instance de l'objet XMLHttpRequest et nous ouvrons une requête en utilisant la méthode open(). Nous passons la méthode HTTP (GET), l'URL de l'API et true pour indiquer que la requête doit être asynchrone.

Nous définissons ensuite une fonction à exécuter lorsque la requête est terminée, en utilisant la propriété onload.

Dans cette fonction, nous vérifions le code de statut HTTP pour déterminer si la requête a réussi (200 OK) ou non. Si la requête a réussi, nous utilisons la méthode JSON.parse() pour convertir la réponse en un objet JavaScript.

Enfin, nous utilisons la réponse pour effectuer une action spécifique, ou nous gérons une erreur si la requête n'a pas réussi.

Bien que XMLHttpRequest soit une méthode plus ancienne pour effectuer des appels à des API, elle peut toujours être utile dans certains cas, comme pour la compatibilité avec les anciens navigateurs.

Cependant, la méthode fetch() est maintenant largement utilisée pour effectuer des appels à des API, car elle est plus simple et plus expressive.

LA MINUTE DE CODE

FETCH

JS

La méthode fetch() est une méthode standard du navigateur qui permet de récupérer des ressources à partir d'un serveur, y compris des données JSON, XML ou HTML. Elle est souvent utilisée pour effectuer des appels à des API en JavaScript.

Voici un exemple d'appel à une API en utilisant la méthode fetch() :

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => {
    // Faire quelque chose avec les données
  })
  .catch(error => {
    // Gérer l'erreur
  });
```

Dans cet exemple, nous appelons l'API en utilisant la méthode fetch() en passant l'URL de l'API en tant qu'argument.

La méthode fetch() renvoie une promesse qui sera résolue avec l'objet Response représentant la réponse HTTP.

Nous utilisons la méthode then() pour traiter la réponse et extraire les données en utilisant la méthode json(). Cette méthode renvoie également une promesse qui sera résolue avec les données JSON.

Ensuite, nous utilisons la deuxième méthode then() pour traiter les données.

FETCH

Nous pouvons faire quelque chose avec les données, comme les afficher sur la page ou les utiliser pour effectuer une action spécifique.

Enfin, nous utilisons la méthode catch() pour gérer les erreurs éventuelles qui pourraient se produire pendant l'appel à l'API.

Il est important de noter que la méthode fetch() utilise le protocole HTTP par défaut, donc si vous utilisez une API avec un protocole différent (comme HTTPS), vous devez spécifier le protocole dans l'URL.

En résumé, la méthode fetch() est une méthode standard du navigateur qui permet de récupérer des ressources à partir d'un serveur, y compris des données JSON, XML ou HTML. Elle est souvent utilisée pour effectuer des appels à des API en JavaScript.



laminutedecode.io

@LAMINUTEDECODE

